For example, you could write a pattern that matches a string containing the text *Ruby* or the text *Rust* using the following regular expression:

```
/Ruby|Rust/
```

The forward slashes delimit the pattern, which consists of the two things we are matching, separated by a pipe character (|). In regular expressions, the pipe character means "either the thing on the right or the thing on the left," in this case either *Ruby* or *Rust*. You can use parentheses within patterns, just as you can in arithmetic expressions, so this pattern matches the same set of strings:

```
/Ru(by|st)/
```

You can also specify *repetition* within patterns. `/ab+c/` matches a string containing an *a* followed by one or more *b*'s, followed by a *c*. Change the plus to an asterisk, and `/ab*c/` creates a regular expression that matches one *a*, zero or more *b*'s, and one *c*.

You can also match one of a group of characters within a pattern. Some common examples are *character classes* such as `\s`, which matches a whitespace character (space, tab, newline, and so on); `\d`, which matches any digit; and `\w`, which matches any character that may appear in a typical word. A dot (.) matches (almost) any character. A table of these character classes appears in Table 2, Character class abbreviations, on page 136.

We can put all this together to produce some useful regular expressions:

```
/\d\d:\d\d:\d\d/     # a time such as 12:34:56
/Ruby.*Rust/       # Ruby, zero or more other chars, then Rust
/Ruby Rust/        # Ruby, exactly one space, and Rust
/Ruby *Rust/       # Ruby, zero or more spaces, and Rust
/Ruby +Rust/       # Ruby, one or more spaces, and Rust
/Ruby\s+Rust/      # Ruby, one or more whitespace characters, then Rust
/Java (Ruby|Rust)/ # Java, a space, and either Ruby or Rust
```

Once you have created a pattern, it seems a shame not to use it. The match operator `=~` can be used to match a string against a regular expression. If the pattern is found in the string, `=~` returns its starting position; otherwise, it returns `nil`. This means you can use regular expressions as the condition in `if` and `while` statements. For example, the following code fragment writes a message if a string contains the text *Ruby* or *Rust*:

```
line = gets
if line =~ /Ruby|Rust/
  puts "Programming language mentioned: #{line}"
end
```

Both strings and regular expressions have a `match?` method which is synonymous to the `=~` operator:

```
line = gets
if line.match?(/Ruby|Rust/)
  puts "Scripting language mentioned: #{line}"
end
```

The `match?` form is probably more common in written Ruby.

The part of a string matched by a regular expression can be replaced with different text using one of Ruby's substitution methods: